

BOOK 10 — ADVANCED JAVA / BACKEND FUNDAMENTALS

Servlets, HTTP, REST, Security Concepts to Spring Boot (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 10 of 24 **Java Versions**

Covered: Servlet API 4.x/5.x (Jakarta EE namespace note included), core concepts version-agnostic

Prerequisites: Book 04 (exceptions), Book 05 (Collections), Book 07 (Streams/records for JSON mapping), Book 09 (JDBC — the data layer these servlets will eventually call) **Next Book:**

11_Spring_Core.md

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: ఇప్పటివరకు మనం console applications రాశాము. ఈ పుస్తకం Java backend web development కి bridge — HTTP protocol, Servlets (Spring కి ముందు, దాని పునాది), REST principles, JSON, మరియు authentication/authorization concepts నేర్చుకుంటాము. Book 11-12 (Spring Core/Boot) ఈ concepts మీదనే build అవుతాయి — Spring "మాయ" కాదు, ఇక్కడ నేర్చుకున్నదాన్ని automate చేస్తుంది.

English: So far we've written console applications. This book bridges to Java backend web development — the HTTP protocol, Servlets (the foundation Spring itself is built on), REST principles, JSON, and authentication/authorization concepts. Books 11-12 (Spring Core/Boot) build directly on these concepts — Spring isn't magic, it automates exactly what you'll learn to do manually here.

Learning Objectives

1. Understand the HTTP protocol: methods, status codes, headers, request/response cycle.
 2. Understand Servlet lifecycle and write basic servlets.
 3. Understand sessions, cookies, and state management over stateless HTTP.
 4. Understand Filters and Listeners for cross-cutting request processing.
 5. Understand REST architectural principles.
 6. Understand JSON serialization/deserialization and Java's `Serializable`.
 7. Understand authentication concepts (how identity is verified).
 8. Understand authorization concepts (how access is controlled).
 9. Understand layered backend architecture patterns.
-

Table of Contents

Ch.	Title
1	HTTP Protocol Fundamentals
2	Servlets: Lifecycle & Basic Structure
3	Sessions & Cookies
4	Filters & Listeners
5	REST Architecture Principles
6	JSON & Serialization
7	Authentication Concepts
8	Authorization Concepts
9	Backend Architecture Patterns
10	Mini Project — Servlet-Based Task Manager
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — HTTP Protocol Fundamentals

Telugu Explanation

HTTP (HyperText Transfer Protocol) అనేది client-server communication కి **stateless**, text-based protocol — ప్రతి request, server కి previous requests గురించి ఏమీ "గుర్తు" లేకుండా, స్వతంత్రంగా handle అవుతుంది. Request methods: GET (data fetch, side-effect లేకుండా), POST (create/submit), PUT (full update/replace), PATCH (partial update), DELETE (remove). Status codes: 2xx (success), 3xx (redirect), 4xx (client error), 5xx (server error).

Professional English Explanation

HTTP is a **stateless**, text-based protocol for client-server communication — each request is handled independently, with the server retaining no memory of previous requests by default. Key methods: GET (fetch data, no side effects — idempotent and safe), POST (create/submit, not idempotent), PUT (full replace, idempotent), PATCH (partial update), DELETE (remove, idempotent). Status code ranges: 2xx (success), 3xx (redirection), 4xx (client error), 5xx (server error).

Diagram — HTTP Request/Response Cycle



Status Code Reference Table

Code	Meaning	Common Use
200 OK	Success	Successful GET/PUT/PATCH
201 Created	Resource created	Successful POST creating a resource
204 No Content	Success, no body	Successful DELETE
400 Bad Request	Client sent invalid data	Validation failure
401 Unauthorized	Not authenticated	Missing/invalid credentials (Ch.7)
403 Forbidden	Authenticated but not allowed	Insufficient permissions (Ch.8)

Code	Meaning	Common Use
404 Not Found	Resource doesn't exist	Invalid ID/URL
409 Conflict	Request conflicts with current state	Duplicate resource, version conflict
500 Internal Server Error	Unhandled server-side failure	Bugs, unexpected exceptions (Book 04)
503 Service Unavailable	Server temporarily can't handle requests	Overload, maintenance, downstream failure (Book 16)

Java Code — A Raw HTTP Client (Java 11+, Book 07 Ch.12)

```
import java.net.URI;
import java.net.http.*;

public class HttpFundamentalsDemo {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient(); // Java 11's
        built-in client (Book 07, Ch.12)

        HttpRequest getRequest = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/get"))
            .GET()
            .header("Accept", "application/json")
            .build();

        HttpResponse<String> response = client.send(getRequest,
            HttpResponse.BodyHandlers.ofString());
        System.out.println("Status code: " + response.statusCode());
        System.out.println("Content-Type header: " + response.headers().firstValue("content-
            type").orElse("none"));

        HttpRequest postRequest = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/post"))
            .POST(HttpRequest.BodyPublishers.ofString("{\"name\":\"Ravi\"}"))
            .header("Content-Type", "application/json")
            .build();

        HttpResponse<String> postResponse = client.send(postRequest,
            HttpResponse.BodyHandlers.ofString());
        System.out.println("POST status: " + postResponse.statusCode());
    }
}
```

Output (illustrative — requires network access)

```
Status code: 200
Content-Type header: application/json
POST status: 200
```

Internal Working

- **Idempotency** matters practically: GET / PUT / DELETE are defined to be idempotent (repeating the same request produces the same end state — calling DELETE /orders/42 twice leaves the order deleted either way, no error on the second call in well-designed APIs), while POST typically is **not** (repeating it may create a duplicate resource) — this distinction directly informs safe client-side retry logic (Book 16's resilience patterns rely heavily on knowing whether a retried request is safe).
- HTTP headers carry metadata separately from the body — Content-Type tells the receiver how to parse the body (Ch.6's JSON), Authorization carries credentials (Ch.7), Accept tells the server what response format the client wants — this separation of "envelope" (headers) from "payload" (body) is a recurring pattern across many protocols, not unique to HTTP.
- Despite HTTP itself being stateless, real applications need to maintain state across requests (a logged-in user, a shopping cart) — this is exactly the problem Ch.3's sessions/cookies solve, layered on top of HTTP's inherently stateless design rather than changing it.

Real-World Example

Telugu: REST API design లో, status codes సరిగ్గా వాడకపోతే (ఉదా. అన్ని errors కి 200 OK పంపడం, body లో error message తో) — client code (frontend, mobile apps) ఆ error ని సరిగ్గా handle చేయలేదు; ఇది real API design mistake, tooling/monitoring కూడా break అవుతుంది. English: A common real API design mistake is returning 200 OK for every response (including errors, signaled only via a body field) instead of proper status codes — this breaks standard client error-handling logic, monitoring/alerting tools that key off status codes, and violates what every HTTP-aware tool in the ecosystem expects.

Interview Answer

"HTTP is a stateless, text-based request-response protocol. Methods have defined semantics — GET/PUT/DELETE are idempotent, POST typically isn't, which matters for safe retry logic. Status codes are grouped by first digit: 2xx success, 3xx redirect, 4xx client error, 5xx server error — using them correctly (not just always returning 200) is essential for clients, monitoring, and tooling across the ecosystem to work correctly."

Cross Questions

- Q: Why does idempotency matter for retry logic? → A: A client can safely retry an idempotent request (GET/PUT/DELETE) after a timeout without risking duplicate side effects; retrying a non-idempotent POST blindly risks creating duplicate resources (e.g., double-charging a payment).
- Q: What's the difference between 401 and 403? → A: 401 means the request lacks valid authentication (who are you?); 403 means the request is authenticated but the identified user lacks permission for this specific action (I know who you are, but you can't do this) — Ch.7/8 cover this distinction in depth.
- Q: Is HTTP itself capable of maintaining state between requests? → A: No — HTTP is stateless by design; state (login sessions, shopping carts) is layered on top via mechanisms like cookies/sessions (Ch.3), tokens (Ch.7), or client-side state.

Tricky Questions

- Q: Is `POST` always non-idempotent in every real API? → A: Not strictly guaranteed by HTTP semantics — well-designed APIs sometimes make specific POST endpoints idempotent (e.g., via an idempotency key the client supplies), but this isn't the HTTP-defined default; assume non-idempotent unless the specific API documents otherwise.
- Q: If a server returns `500 Internal Server Error`, does that tell the client anything about whether retrying is safe? → A: Not on its own — a 500 could result from a transient issue (safe-ish to retry, especially for idempotent methods) or from a bug that will fail identically every time (retrying is pointless); this ambiguity is exactly why resilience patterns (Book 16) combine status codes with additional signals (retry-after headers, circuit breaker state) rather than relying on the status code alone.

Coding Exercise

L1: Use `HttpClient` to send a GET request to a public test API and print the status code and response body. **L2:** Send a POST request with a JSON body and inspect the response. **L3:** Write out, from memory, the meaning of all 10 status codes in the reference table, then verify against the table. **L4 (Interview):** Explain idempotency and why it matters for retry logic, with an example of a non-idempotent operation. **L5 (Senior):** Review an API returning `200 OK` for all responses (errors included, via a body flag) — explain the practical problems this causes for clients and monitoring. **L6 (Mastery):** Explain, from memory, why HTTP is described as stateless, and name 2 mechanisms used to layer state on top of it.

CHAPTER 2 — Servlets: Lifecycle & Basic Structure

Telugu Explanation

Servlet అనేది Java class, HTTP requests handle చేయడానికి, Servlet Container (Tomcat వంటివి) లో run అవుతుంది. Servlet lifecycle: `init()` (ఒక్కసారి, container load చేసినప్పుడు), `service()` (ప్రతి request కి — `doGet()` / `doPost()` /etc కి dispatch చేస్తుంది HTTP method బట్టి), `destroy()` (ఒక్కసారి, container shutdown అయ్యేటప్పుడు). Spring MVC (Book 12) ఈ servlet model మీదనే build అయ్యింది — `DispatcherServlet` ఒక్కటే central servlet గా అన్ని requests ని handle చేస్తుంది.

Professional English Explanation

A servlet is a Java class handling HTTP requests, running inside a Servlet Container (like Tomcat). Lifecycle: `init()` (once, when the container loads it), `service()` (once per request — dispatches to `doGet()` / `doPost()` /etc. based on HTTP method), `destroy()` (once, on container shutdown). Spring MVC (Book 12) is built directly on this servlet model — a single `DispatcherServlet` acts as the central servlet handling all requests, routing them to your `@Controller` methods.

Java Code

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.io.PrintWriter;

public class TaskServlet extends HttpServlet {

    @Override
    public void init() throws ServletException {
        System.out.println("TaskServlet initialized - runs ONCE when container loads it");
        // Typical use: initialize a DB connection pool (Book 09, Ch.7) reference, load config
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        resp.setContentType("application/json");
        String taskId = req.getParameter("id"); // query param: ?id=42

        try (PrintWriter out = resp.getWriter()) {
            if (taskId == null) {
                out.println("{\"tasks\": [\"Buy groceries\", \"Finish report\"]}");
            } else {
                resp.setStatus(HttpServletResponse.SC_OK);
                out.println("{\"id\": \"" + taskId + "\", \"title\": \"Sample Task\"}");
            }
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException
    {
        String title = req.getParameter("title");
        if (title == null || title.isBlank()) {
            resp.sendError(HttpServletResponse.SC_BAD_REQUEST, "title is required"); // 400
            return;
        }
        resp.setStatus(HttpServletResponse.SC_CREATED); //
201
        resp.setContentType("application/json");
        try (PrintWriter out = resp.getWriter()) {
            out.println("{\"id\": \"99\", \"title\": \"" + title + "\", \"status\": \"created\"}");
        }
    }

    @Override
    public void destroy() {
        System.out.println("TaskServlet destroyed - runs ONCE when container shuts down");
        // Typical use: release resources, close connection pools
    }
}
```


web.xml / Annotation-Based Registration

```
// Modern annotation-based registration (Servlet 3.0+), instead of web.xml
@jakarta.servlet.annotation.WebServlet("/tasks")
public class AnnotatedTaskServlet extends HttpServlet {
    // same doGet/doPost logic as above
}
```

Internal Working

- The Servlet Container (Tomcat, Jetty) creates **exactly one instance** of each servlet class by default and reuses it across **all** requests, calling `service()` (which dispatches to `doGet` / `doPost` /etc.) on that shared instance concurrently from multiple threads — this has a critical, direct consequence: **instance fields on a servlet are shared, mutable state across concurrent requests**, exactly the race-condition risk from Book 08, Part 1, Ch.3, unless properly synchronized or, better, avoided entirely by keeping servlets stateless and putting any needed state in method-local variables, `HttpServletRequest` attributes, or the session (Ch.3).
- `init()` and `destroy()` running exactly once each (not per-request) is precisely why they're the correct place for expensive one-time setup/teardown (Book 09, Ch.7's connection pool initialization is a classic example) — mirroring the same "expensive resource, reuse don't recreate" principle from connection pooling and thread pooling (Book 08, Ch.1).
- This chapter's raw servlet code — parsing request parameters, manually building JSON strings, setting status codes — is **exactly** what Spring MVC's `@RestController` (Book 12) automates for you: `@GetMapping`, `@RequestParam`, and automatic JSON serialization (Ch.6) all exist specifically to eliminate this boilerplate while still ultimately running on top of a servlet (`DispatcherServlet`) underneath.

Real-World Example

Telugu: Spring Boot application startup అయినప్పుడు, embedded Tomcat ఒక్క `DispatcherServlet` register చేస్తుంది — మీ `@RestController` classes direct servlets కాదు, `DispatcherServlet` వాటికి requests route చేస్తుంది. ఈ chapter అర్థం చేసుకోవడం, Book 12 లో "Spring ఎలా పనిచేస్తుంది" అనే మాయని తొలగిస్తుంది. English: When a Spring Boot application starts, its embedded Tomcat registers exactly one `DispatcherServlet` — your `@RestController` classes aren't servlets themselves; `DispatcherServlet` routes requests to them. Understanding this chapter is precisely what demystifies "how Spring actually works" in Book 12, rather than treating it as unexplained magic.

Interview Answer

"A servlet handles HTTP requests inside a container, with a lifecycle of `init()` (once), `service()` / `doGet()` / `doPost()` (per request), and `destroy()` (once). Critically, the container reuses a single servlet instance across all concurrent requests, so instance fields are shared mutable state across threads — servlets should be kept stateless, storing per-request data in method locals, request attributes, or the session rather than instance fields. Spring MVC's `DispatcherServlet` is itself a servlet, routing requests to your `@Controller` methods."

Cross Questions

- Q: Is a new servlet instance created for every request? → A: No — by default, one instance is created and reused across all requests and all threads, which is why servlet instance state must be handled carefully (or avoided).
- Q: Where should per-request data be stored instead of instance fields? → A: Method-local variables (thread-safe by definition, Book 08 Part 1, Ch.1), `HttpServletRequest` attributes (scoped to that one request), or the session (Ch.3, scoped to that one user across multiple requests) — never servlet instance fields for anything request-specific.
- Q: What is `DispatcherServlet`, and how does it relate to plain servlets? → A: It's Spring MVC's single, central servlet (Book 12) that receives all incoming requests and routes them to the appropriate `@Controller` / `@RestController` method — it IS a servlet, built using exactly this chapter's underlying model.

Tricky Questions

- Q: If a servlet has a non-final instance field incremented in `doGet()` (like a naive request counter), is this safe? → A: No — this is a direct instance of Book 08, Part 1, Ch.3's race condition, since the single shared servlet instance handles concurrent requests on different threads; it needs proper synchronization or an atomic type (Book 08, Ch.7), or should be redesigned to avoid shared mutable instance state entirely.
- Q: Does `init()` running once mean it's safe to skip synchronization for state it sets up? → A: The setup itself (running once, before any request-handling begins) is safe, but if that setup produces a *mutable* object later modified during request handling, that later modification still needs the same care as any other shared mutable servlet state.

Coding Exercise

L1: Write a servlet with `doGet()` returning a static JSON message, and deploy/test it (via a local Tomcat or embedded servlet container, if available in your environment). **L2:** Add `doPost()` handling with request parameter validation and proper status codes (400 for bad input, 201 for success). **L3:** Deliberately add a non-thread-safe instance field counter, and explain (in comments) the race condition risk without necessarily needing to reproduce it under real concurrent load. **L4 (Interview):** Explain the servlet lifecycle and why instance fields are risky. **L5 (Senior):** Explain how Spring Boot's embedded Tomcat and `DispatcherServlet` relate to the raw servlet concepts in this chapter. **L6 (Mastery):** Explain, from memory, why a single shared servlet instance handling concurrent requests is analogous to Book 08's shared mutable state risks.

CHAPTER 3 — Sessions & Cookies

Telugu Explanation

HTTP stateless కాబట్టి, "ఈ request, గత request చేసిన అదే user నుండి వచ్చిందా?" అని తెలుసుకోవడానికి **Cookies** వాడతారు — server ఒక unique session ID ని cookie గా client కి పంపుతుంది, client ప్రతి తర్వాతి request తో ఆ cookie తిరిగి పంపుతుంది. **HttpSession** server-side లో ఈ session ID కి associated data (login state, cart items) store చేస్తుంది.

Professional English Explanation

Since HTTP is stateless, **cookies** solve "is this request from the same user as a previous request?" — the server sends a unique session ID as a cookie, and the client automatically sends that cookie back on every subsequent request. **HttpSession** stores server-side data (login state, cart contents) associated with that session ID.

Java Code

```
import jakarta.servlet.http.*;
import java.io.PrintWriter;

public class SessionCookieServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
    java.io.IOException {
        HttpSession session = req.getSession(true);           // true = create if it
        doesn't exist
        System.out.println("Session ID: " + session.getId());
        System.out.println("Is new session? " + session.isNew());

        Integer visitCount = (Integer) session.getAttribute("visitCount");
        visitCount = (visitCount == null) ? 1 : visitCount + 1;
        session.setAttribute("visitCount", visitCount);       // server-side state
        tied to this session

        session.setMaxInactiveInterval(30 * 60);              // 30 minutes idle
        timeout

        // A cookie is also set MANUALLY here to show the underlying mechanism (JSESSIONID is
        set automatically for sessions)
        Cookie preferenceCookie = new Cookie("theme", "dark");
        preferenceCookie.setMaxAge(7 * 24 * 60 * 60);           // 7 days, in
        seconds
        preferenceCookie.setHttpOnly(true);                    // JavaScript
        CANNOT read this cookie - XSS mitigation
        preferenceCookie.setSecure(true);                       // only sent
        over HTTPS
        resp.addCookie(preferenceCookie);

        resp.setContentType("application/json");
        try (PrintWriter out = resp.getWriter()) {
            out.println("{\"sessionId\": \"" + session.getId() + "\", \"visitCount\": " +
            visitCount + "\"}");
        }
    }
}
```

Sequence Diagram — First Visit vs Return Visit

FIRST VISIT:

```
Client ->> Server: GET /page (no cookie sent)
Server ->> Client: 200 OK, Set-Cookie: JSESSIONID=ABC123
                    (server creates a new HttpSession internally, keyed by ABC123)
```

RETURN VISIT (same browser, cookie still valid):

```
Client ->> Server: GET /page, Cookie: JSESSIONID=ABC123
Server: looks up HttpSession by ABC123 -> finds the SAME session, with visitCount=1 already
stored
Server ->> Client: 200 OK, {"visitCount": 2}
```

Internal Working

- The session ID cookie (conventionally `JSESSIONID`) is set **automatically** by the container the first time `req.getSession()` is called; it's fundamentally just a correlation key — the actual session **data** (`visitCount`, login state, etc.) lives entirely in server-side memory (or a distributed session store in clustered deployments), keyed by that ID, never sent to the client itself.
- `HttpOnly` and `Secure` cookie flags are real, important security controls: `HttpOnly` prevents client-side JavaScript from reading the cookie at all (mitigating cookie-theft via Cross-Site Scripting/XSS attacks), and `Secure` ensures the cookie is only ever transmitted over HTTPS, never plaintext HTTP — both are standard, expected practice for any session-identifying cookie in production.
- **Session state doesn't scale trivially** across multiple server instances — if a load balancer routes a return request to a *different* server than the one that created the session, that server won't have the in-memory session data unless something (sticky sessions at the load balancer, or a shared/distributed session store like Redis) addresses this — a real architectural concern directly connecting to Book 21's system design "stateless vs stateful" and "sticky sessions" discussion.

Real-World Example

Telugu: Login state maintain చేయడానికి — user login చేసినప్పుడు, `session.setAttribute("userId", ...)` పెడతారు; తర్వాతి requests అన్నీ ఆ session ద్వారా "ఈ user login అయ్యాడు" అని తెలుసుకుంటాయి. Modern microservices (Book 16) చాలావరకు session-based auth బదులు **stateless JWT tokens** (Ch.7) వాడతారు — precisely సర్వర్ instances మధ్య session-sharing complexity avoid చేయడానికి. English: Maintaining login state is the classic session use case — on login, `session.setAttribute("userId", ...)` is set; subsequent requests check the session to know the user is authenticated. Modern microservices (Book 16) increasingly favor **stateless JWT tokens** (Ch.7) over session-based auth precisely to avoid the cross-server session-sharing complexity described above — a genuine, deliberate architectural trade-off you'll see debated in real system design.

Interview Answer

"Cookies let a stateless HTTP server recognize repeat requests from the same client, typically via a session ID cookie. `HttpSession` stores the actual server-side state (login status, cart) keyed by that ID. `HttpOnly` and `Secure` cookie flags are essential security controls preventing XSS-based cookie theft and plaintext transmission. Session state doesn't trivially scale across multiple server instances without sticky sessions or a shared session store — which is a major reason modern stateless architectures increasingly prefer token-based auth (Ch.7) instead."

Cross Questions

- Q: Is any actual user data transmitted inside the session cookie itself? → A: No — the cookie carries only an opaque session ID; the real data lives server-side (or in a distributed store), looked up by that ID.
- Q: What does the `HttpOnly` flag protect against? → A: Client-side JavaScript reading the cookie, mitigating cookie theft via Cross-Site Scripting (XSS) attacks.

- Q: Why do multi-server deployments need special handling for sessions? → A: Because session data by default lives only in the memory of whichever server instance created it — a request routed to a different instance won't find it, unless sticky sessions or a shared/distributed session store address this.

Tricky Questions

- Q: If a user disables cookies in their browser, can `HttpSession`-based state tracking still work at all? → A: Not via the standard cookie mechanism — servlet containers can fall back to URL rewriting (encoding the session ID into the URL itself, `;jsessionid=...`), though this is rarely used in modern applications and has its own security/usability drawbacks (session IDs leaking into browser history, shared links, etc.).
- Q: Does `session.setMaxInactiveInterval()` measure time since the session was `CREATED` or time since the `LAST` request? → A: Time since the last request (activity) — it's an *inactivity* timeout, resetting on every request that touches the session, not a fixed absolute expiry from creation time.

Coding Exercise

L1: Build a servlet tracking and displaying a per-session visit counter across multiple requests (same browser). **L2:** Set a custom cookie with `HttpOnly`, `Secure`, and a max-age, and inspect it in browser dev tools. **L3:** Research (via documentation) how your servlet container's session timeout is configured, and set it explicitly. **L4 (Interview):** Explain what `HttpOnly` and `Secure` cookie flags protect against. **L5 (Senior):** Design a session-sharing strategy for a load-balanced, multi-instance deployment (sticky sessions vs a shared Redis-backed session store), justifying your choice. **L6 (Mastery):** Explain, from memory, why modern microservices increasingly prefer stateless token-based auth over session-based auth.

CHAPTER 4 — Filters & Listeners

Telugu Explanation

Filter అనేది request/response ని servlet కి చేరకముందు (లేదా తర్వాత) intercept చేసే component — cross-cutting concerns (logging, authentication check, compression) కి ideal, business logic నుండి వేరుగా.

Listener అనేది application/session/request lifecycle events (startup, session created, attribute added) కి react చేసే component.

Professional English Explanation

A **Filter** intercepts requests/responses before (or after) they reach a servlet — ideal for cross-cutting concerns (logging, authentication checks, compression) kept separate from business logic. A

Listener reacts to application/session/request lifecycle events (startup, session creation, attribute changes).

Java Code — Filter

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

@jakarta.servlet.annotation.WebFilter("/*") // applies to ALL
requests
public class LoggingAuthFilter implements Filter {

    @Override
    public void init(FilterConfig config) { System.out.println("LoggingAuthFilter
initialized"); }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest httpReq = (HttpServletRequest) request;
        HttpServletResponse httpResp = (HttpServletResponse) response;

        long start = System.currentTimeMillis();
        System.out.println("--> Incoming: " + httpReq.getMethod() + " " +
httpReq.getRequestURI());

        String authHeader = httpReq.getHeader("Authorization");
        if (httpReq.getRequestURI().startsWith("/api/secure") && authHeader == null) {
            httpResp.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Missing Authorization
header");
            return; // SHORT-CIRCUIT
- chain.doFilter() NOT called
        }

        chain.doFilter(request, response); // pass
control to the NEXT filter/servlet

        long duration = System.currentTimeMillis() - start;
        System.out.println("<-- Completed: " + httpReq.getRequestURI() + " in " + duration +
"ms, status="
            + httpResp.getStatus());
    }

    @Override
    public void destroy() { System.out.println("LoggingAuthFilter destroyed"); }
}
```


Java Code — Listener

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class AppLifecycleListener implements ServletContextListener, HttpSessionListener {

    @Override
    public void contextInitialized(ServletContextEvent sce) {
        System.out.println("Application STARTING UP - initialize shared resources here (e.g.,
connection pool, Book 09 Ch.7)");
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        System.out.println("Application SHUTTING DOWN - release shared resources here");
    }

    @Override
    public void sessionCreated(HttpSessionEvent se) {
        System.out.println("New session created: " + se.getSession().getId());
    }

    @Override
    public void sessionDestroyed(HttpSessionEvent se) {
        System.out.println("Session destroyed/expired: " + se.getSession().getId());
    }
}
```

Diagram — Filter Chain

```
Request --> [Filter 1: Logging] --> [Filter 2: Auth check] --> [Filter 3: Compression] -->
Servlet

          |               |               |
      chain.doFilter()   chain.doFilter()   chain.doFilter()
      (or short-circuit   (or short-circuit   (or short-circuit
        by NOT calling it) by NOT calling it) by NOT calling it)

Response <-- [Filter 1: log duration] <-- [Filter 2: no-op] <-- [Filter 3: compress body] <--
Servlet response
```

Internal Working

- Filters form a **chain** — each filter decides whether to call `chain.doFilter()` (passing control onward to the next filter or, eventually, the servlet) or to **short-circuit** (handle the response itself and return, as the auth check does above for missing credentials) — this exact chain-of-responsibility shape (Book 18) is precisely what Spring Security's filter chain (Book 14) is built on, and what Spring MVC interceptors conceptually mirror at a higher level.
- Code *after* `chain.doFilter()` in a filter's `doFilter()` method runs on the way **back out** (once the servlet, and any inner filters, have finished) — this is why the logging filter can measure total request duration and log the final response status, by placing that logic after the `chain.doFilter()` call.

- `ServletContextListener` fires exactly once at application startup/shutdown — the direct servlet-API analog to a Spring `@PostConstruct / ApplicationListener<ContextRefreshedEvent>` (Book 11), and the correct place for the kind of one-time resource setup also seen in Ch.2's `init()`, but scoped to the whole application rather than one specific servlet.

Real-World Example

Telugu: Real backend applications లో logging, authentication, CORS handling, request/response compression — అన్నీ filters నా implement చేస్తారు, ప్రతి servlet/controller లో duplicate చేయకుండా. Spring Security (Book 14) దీని meీదనే build అయ్యింది — దాని "security filter chain" ఖచ్చితంగా ఈ chapter యొక్క filter chain concept. English: Real backend applications implement logging, authentication, CORS handling, and response compression as filters — cross-cutting concerns applied once, centrally, instead of duplicated in every servlet/controller. Spring Security (Book 14) is built directly on this exact mechanism — its "security filter chain" is precisely this chapter's filter chain concept, just configured declaratively.

Interview Answer

"Filters intercept requests/responses around servlet processing, ideal for cross-cutting concerns like logging, auth checks, and compression, kept separate from business logic — they form a chain where each filter can pass control onward via `chain.doFilter()` or short-circuit by not calling it. Listeners react to lifecycle events — `ServletContextListener` for application startup/shutdown, `HttpSessionListener` for session creation/destruction. Spring Security's filter chain is built directly on this same mechanism."

Cross Questions

- Q: What happens if a filter never calls `chain.doFilter()`? → A: The request never reaches the next filter or the servlet — the filter has fully short-circuited the chain, typically because it's rejecting the request (as the auth check does for missing credentials).
- Q: Where should code go if it needs to run after the servlet has produced its response (e.g., to log the final status code)? → A: After the `chain.doFilter()` call within the filter's own `doFilter()` method — that code runs on the way back out, once everything downstream has completed.
- Q: What's the difference between a Filter and a Listener? → A: A Filter actively participates in the request/response processing chain for matching requests; a Listener passively reacts to lifecycle events (application start/stop, session create/destroy) without being part of the request-processing chain itself.

Tricky Questions

- Q: Can multiple filters be registered for the same URL pattern, and if so, what determines their order? → A: Yes — order is typically determined by registration order (annotation processing order, or explicit ordering configuration in `web.xml` / newer mechanisms), which is why explicit, deliberate ordering matters for correctness (e.g., an auth filter should generally run before a business-logic-adjacent filter).

- Q: Does a `ServletContextListener`'s `contextInitialized()` run before or after any servlet's own `init()`? → A: Before — application-level initialization (`contextInitialized()`) happens as part of application startup, ahead of individual servlets being initialized (which may happen lazily, on first request, unless configured for eager `load-on-startup` initialization).

Coding Exercise

L1: Write a logging filter that prints method, URI, and duration for every request. **L2:** Add an authentication-check filter that short-circuits with 401 for requests missing a required header on `/api/secure/*` paths. **L3:** Write a `ServletContextListener` printing startup/shutdown messages. **L4 (Interview):** Explain the filter chain concept and what "short-circuiting" means in this context. **L5 (Senior):** Design a filter chain ordering (logging, CORS, authentication, rate-limiting) for a production API, justifying the order chosen. **L6 (Mastery):** Explain, from memory, why code placed after `chain.doFilter()` runs on the response's way back out, and give a concrete use case for it.

CHAPTER 5 — REST Architecture Principles

Telugu Explanation

REST (Representational State Transfer) అనేది web APIs design చేయడానికి ఒక **architectural style** (specific technology కాదు) — HTTP యొక్క native features (methods, status codes, statelessness) ని fully ఉపయోగించుకుంటుంది. Core principles: **Statelessness** (ప్రతి request తనకే కావాల్సిన సమాచారం అంతా carry చేయాలి, server-side session మేరీద అధారపడకూడదు), **Resource-based URLs** (nouns, verbs కాదు: /orders/42 కాదు /getOrder?id=42), **Uniform Interface** (HTTP methods correct గా వాడటం), **Representation** (JSON/XML ద్వారా resource state exchange).

Professional English Explanation

REST (Representational State Transfer) is an **architectural style** (not a specific technology) for designing web APIs, fully leveraging HTTP's native features (methods, status codes, statelessness). Core principles: **Statelessness** (every request carries all information needed to process it, not relying on server-side session state), **Resource-based URLs** (nouns, not verbs: /orders/42 , not /getOrder?id=42), **Uniform Interface** (correct use of HTTP methods for their defined semantics), and **Representation** (exchanging resource state via JSON/XML).

Java Code — RESTful vs Non-RESTful Design

```
public class RestPrinciplesDemo {
    public static void main(String[] args) {
        System.out.println("--- NON-RESTful design (verb-based URLs, misused methods) ---");
        System.out.println("GET /getOrder?id=42 -- verb in URL, should be a noun-based resource path");
        System.out.println("POST /deleteOrder?id=42 -- wrong method - DELETE exists for exactly this");
        System.out.println("POST /updateOrderStatus?id=42&s=SHIPPED -- wrong method - PUT/PATCH exists for exactly this");

        System.out.println("\n--- RESTful design (resource-based URLs, correct HTTP methods) ---");
        System.out.println("GET /orders/42 -- fetch order 42");
        System.out.println("GET /orders status=SHIPPED for filtering) -- fetch all orders (optionally ?");
        System.out.println("POST /orders -- create a new order");
        System.out.println("PUT /orders/42 -- fully replace order 42");
        System.out.println("PATCH /orders/42 -- partially update order 42 (e.g., just the status)");
        System.out.println("DELETE /orders/42 -- delete order 42");
        System.out.println("GET /orders/42/items -- nested resource: items belonging to order 42");

        System.out.println("\n--- Statelessness example ---");
        System.out.println("BAD: server remembers 'last searched category' from a previous request in session state");
        System.out.println("GOOD: client sends GET /products?category=electronics&page=2 - fully self-describing request");
    }
}
```

Output

```
--- NON-RESTful design (verb-based URLs, misused methods) ---
GET  /getOrder?id=42          -- verb in URL, should be a noun-based resource path
POST /deleteOrder?id=42       -- wrong method - DELETE exists for exactly this
POST /updateOrderStatus?id=42&s=SHIPPED -- wrong method - PUT/PATCH exists for exactly this

--- RESTful design (resource-based URLs, correct HTTP methods) ---
GET  /orders/42              -- fetch order 42
GET  /orders                  -- fetch all orders (optionally ?status=SHIPPED for filtering)
POST /orders                  -- create a new order
PUT  /orders/42              -- fully replace order 42
PATCH /orders/42            -- partially update order 42 (e.g., just the status)
DELETE /orders/42            -- delete order 42
GET  /orders/42/items        -- nested resource: items belonging to order 42

--- Statelessness example ---
BAD:  server remembers 'last searched category' from a previous request in session state
GOOD: client sends GET /products?category=electronics&page=2 - fully self-describing request
```

Internal Working

- REST's **statelessness** principle is a deliberate architectural constraint, not an accident of HTTP — it's what makes REST APIs trivially **horizontally scalable** (any server instance can handle any request, since no request depends on server-side memory from a previous one, directly connecting to Ch.3's session-sharing complexity and Book 21's stateless-architecture system design principles) and simpler to cache, debug, and reason about.
- Using HTTP methods per their defined semantics (Ch.1's idempotency table) isn't just stylistic — it lets generic HTTP-aware tooling (caches, proxies, browsers, API gateways) make correct assumptions automatically: a `GET` can be safely cached and retried; a `POST` generally shouldn't be auto-retried by a proxy; a `DELETE` on an already-deleted resource can correctly return success (idempotent) rather than needing special client-side handling.
- REST is explicitly **not** a strict protocol/standard with a formal specification the way, say, SOAP is — it's a set of architectural guidelines (originally from Roy Fielding's dissertation), which is exactly why real-world "REST APIs" vary somewhat in how strictly they follow every principle (fully "RESTful" including HATEOAS is rare in practice; most production APIs are more accurately "HTTP+JSON APIs following most REST conventions").

Real-World Example

Telugu: Real production APIs (Book 12 Spring Boot `@RestController`) ఖచ్చితంగా ఈ resource-based URL + correct HTTP method conventions follow చేస్తాయి — `/api/orders/{id}` వంటి patterns, `@GetMapping` / `@PostMapping` / `@PutMapping` / `@DeleteMapping` annotations తో directly map అవుతాయి.

English: Real production APIs (Book 12's Spring Boot `@RestController`) follow exactly these resource-based URL and correct-HTTP-method conventions — patterns like `/api/orders/{id}` map directly onto `@GetMapping` / `@PostMapping` / `@PutMapping` / `@DeleteMapping` annotations, which is precisely why understanding REST principles here makes Spring MVC's annotation-based routing immediately intuitive rather than arbitrary.

Interview Answer

"REST is an architectural style for web APIs built on HTTP's native features: statelessness (every request self-contained, enabling horizontal scalability), resource-based URLs using nouns not verbs, and correct use of HTTP methods per their defined semantics rather than always using GET/POST for everything. This isn't just style — it lets generic HTTP tooling (caches, proxies, gateways) make correct assumptions automatically."

Cross Questions

- Q: Why does REST's statelessness principle enable horizontal scalability? → A: Since no request depends on server-side memory from a prior request, any server instance behind a load balancer can handle any request — directly avoiding the session-sharing complexity discussed in Ch.3.
- Q: What's wrong with `POST /deleteOrder?id=42` ? → A: It uses the wrong HTTP method (`POST` instead of `DELETE`) and a verb in the URL instead of a resource-based path (`DELETE /orders/42`) — both violate REST conventions and prevent HTTP-aware tooling from reasoning correctly about the request's semantics.
- Q: Is REST a formal, strictly-defined protocol like SOAP? → A: No — it's an architectural style/set of guidelines, not a rigid specification, which is why real-world "REST APIs" vary in strictness; most are more accurately "HTTP+JSON APIs following most REST conventions" rather than fully, formally RESTful.

Tricky Questions

- Q: Does REST forbid ever storing any state on the server at all? → A: No — statelessness specifically means the *server doesn't rely on remembering previous requests to correctly process a new one*; the server can certainly have persistent state (a database, Book 09), it just can't require session-specific context carried over implicitly between requests.
- Q: Is `PATCH /orders/42` with a full replacement body functionally different from `PUT /orders/42` ? → A: In practice, if a client sends a complete representation via `PATCH` , the *effect* may be identical to `PUT` , but the *semantic intent* differs — `PUT` implies "replace entirely, missing fields become absent/default," while `PATCH` implies "apply only these specific changes," which matters for how a well-behaved API should interpret partial vs. complete request bodies.

Coding Exercise

L1: Redesign 5 non-RESTful endpoint examples (verb-based URLs, misused methods) into proper RESTful equivalents. **L2:** Design a resource-based URL scheme for a small blog API (posts, comments, authors), including nested resources. **L3:** Identify which of 6 given operations should be idempotent and which shouldn't, justifying each. **L4 (Interview):** Explain REST's statelessness principle and why it matters for scalability. **L5 (Senior):** Review an existing API using verb-based URLs and POST for everything — propose a RESTful redesign. **L6 (Mastery):** Explain, from memory, why REST is described as an architectural style rather than a strict protocol.

CHAPTER 6 — JSON & Serialization

Telugu Explanation

JSON (JavaScript Object Notation) REST APIs కి de facto standard data format — human-readable, lightweight. Java objects ని JSON గా convert చేయడాన్ని **serialization** అంటారు (libraries: Jackson, Gson); తిరిగి JSON నుండి Java objects కి convert చేయడాన్ని **deserialization** అంటారు. `java.io.Serializable` వేరే concept — JVM-internal binary serialization కి (network transfer, caching, session replication కోసం), JSON కి కాదు.

Professional English Explanation

JSON is the de facto standard data format for REST APIs — human-readable, lightweight. Converting Java objects to JSON is **serialization** (libraries: Jackson, Gson); converting JSON back to Java objects is **deserialization**. `java.io.Serializable` is a distinct concept — for JVM-internal binary serialization (network transfer, caching, session replication), not JSON.

Java Code — Jackson (the standard JSON library, used by default in Spring Boot)

```
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.annotation.*;
import java.time.LocalDate;
import java.util.List;

public class JsonSerializationDemo {

    record Order(
        @JsonProperty("order_id") String orderId,           // customize the JSON
        field name String customerName,
        @JsonFormat(pattern = "yyyy-MM-dd") LocalDate orderDate, // custom date format
        (Book 07, Ch.10) double total,
        @JsonIgnore String internalNotes                    // NEVER serialize
        this field to JSON
    ) {}

    public static void main(String[] args) throws Exception {
        ObjectMapper mapper = new ObjectMapper();
        mapper.findAndRegisterModules();                    // needed for
        LocalDate (Book 07) support

        Order order = new Order("ORD-42", "Ravi Kumar", LocalDate.of(2026, 8, 28), 1499.0,
        "internal audit flag");

        // SERIALIZATION: Java object -> JSON string
        String json = mapper.writeValueAsString(order);
        System.out.println("Serialized JSON: " + json);

        // DESERIALIZATION: JSON string -> Java object
        String incomingJson = ""
            {"order_id": "ORD-99", "customerName": "Anitha", "orderDate": "2026-08-20",
        "total": 750.0}
            "";
        Order deserialized = mapper.readValue(incomingJson, Order.class);
        System.out.println("Deserialized object: " + deserialized);

        // List serialization
        List<Order> orders = List.of(order, deserialized);
        String jsonList = mapper.writerWithDefaultPrettyPrinter().writeValueAsString(orders);
        System.out.println("Pretty-printed list:\n" + jsonList);

        // Handling unknown/extra JSON fields gracefully (common when APIs evolve)

        mapper.configure(com.fasterxml.jackson.databind.DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
        String jsonWithExtraField = "{ \"order_id\": \"ORD-1\", \"customerName\": \"X\",
        \"orderDate\": \"2026-01-01\", \"
            + \"total\": 100.0, \"unexpectedNewField\": \"ignored gracefully\"}";
        Order tolerant = mapper.readValue(jsonWithExtraField, Order.class);
        System.out.println("Tolerant deserialization succeeded: " + tolerant);
    }
}
```


Output

```
Serialized JSON: {"order_id":"ORD-42","customerName":"Ravi Kumar","orderDate":"2026-08-28","total":1499.0}
Deserialized object: Order[orderId=ORD-99, customerName=Anitha, orderDate=2026-08-20, total=750.0, internalNotes=null]
Pretty-printed list:
[ {
  "order_id" : "ORD-42",
  "customerName" : "Ravi Kumar",
  "orderDate" : "2026-08-28",
  "total" : 1499.0
}, {
  "order_id" : "ORD-99",
  "customerName" : "Anitha",
  "orderDate" : "2026-08-20",
  "total" : 750.0
} ]
Note: internalNotes never appears - @JsonIgnore excludes it from serialization entirely
Tolerant deserialization succeeded: Order[orderId=ORD-1, customerName=X, orderDate=2026-01-01, total=100.0, internalNotes=null]
```

Internal Working

- Jackson uses **reflection** (inspecting a class's fields/accessors at runtime) to automatically map between Java object fields and JSON properties by default — this is why records (Book 07, Ch.13) work seamlessly with it out of the box (their accessor methods follow a predictable naming pattern), and why annotations like `@JsonProperty` / `@JsonIgnore` / `@JsonFormat` exist to override that default reflective mapping when the JSON shape needs to differ from the Java class shape (a very common real need — external API contracts rarely match internal naming conventions exactly).
- `@JsonIgnore` is a genuine **security-relevant** annotation, not just a convenience — accidentally serializing an internal field (a password hash, an internal audit flag, an admin-only note) into a public API response is a real, documented category of data-leak vulnerability; deliberately marking sensitive fields is a necessary defensive practice, not an afterthought.
- `FAIL_ON_UNKNOWN_PROPERTIES = false` (disabling strict deserialization) is a deliberate **API evolution** design choice — it lets your application tolerate a producer (an external API, a newer client) sending additional fields your current code doesn't know about yet, without breaking; the alternative (strict, failing on any unknown field) is sometimes preferred instead specifically to catch unexpected data early — a genuine trade-off depending on the API's stability/evolution needs.

Real-World Example

Telugu: REST API `@RestController` (Book 12) methods `List<Order>` return చేస్తే, Spring Boot అంతర్గతంగా Jackson వాడి automatic గా JSON కి serialize చేస్తుంది — మీరు `ObjectMapper` explicit గా వాడాల్సిన అవసరం లేదు, కానీ ఈ chapter అర్థం చేసుకోవడం, ఆ "automatic" behavior వెనుక ఏముందో explain చేస్తుంది. English: When a Spring Boot `@RestController` (Book 12) method returns `List<Order>`, Spring automatically uses Jackson underneath to serialize it to JSON — you rarely touch `ObjectMapper` explicitly in application code, but understanding this chapter is exactly what explains what that

"automatic" behavior is actually doing, including why `@JsonIgnore` on a sensitive field still matters even though you never call Jackson directly yourself.

Interview Answer

"JSON is the standard data format for REST API payloads. Jackson (the library Spring Boot uses by default) handles serialization (Java object → JSON) and deserialization (JSON → Java object) via reflection, with annotations like `@JsonProperty`, `@JsonIgnore`, and `@JsonFormat` customizing the mapping. `@JsonIgnore` is genuinely security-relevant — preventing sensitive internal fields from leaking into API responses. This is distinct from `java.io.Serializable`, which is for JVM-internal binary serialization, not JSON."

Cross Questions

- Q: Why do records (Book 07, Ch.13) work well with Jackson by default? → A: Their auto-generated accessor methods follow a predictable, reflection-discoverable pattern Jackson can map to JSON property names automatically, without needing manual configuration for the common case.
- Q: Why is `@JsonIgnore` more than just a convenience feature? → A: Forgetting to exclude a sensitive internal field (credentials, internal flags) from serialization is a real, documented data-leak vulnerability class — deliberately marking such fields is a necessary security practice.
- Q: What's the difference between Jackson's JSON serialization and `java.io.Serializable`? → A: `Serializable` is for JVM-internal binary object serialization (used for things like session replication or object caching between JVMs); JSON serialization via Jackson is specifically for the text-based, cross-language format used in HTTP API payloads — they solve different problems and aren't interchangeable.

Tricky Questions

- Q: If a JSON payload has a field your Java class doesn't declare at all, what happens by default (with Jackson's default settings)? → A: By default, Jackson throws an exception (`UnrecognizedPropertyException`) unless `FAIL_ON_UNKNOWN_PROPERTIES` is explicitly disabled — many real applications do disable it deliberately for API evolution tolerance, but it's not the out-of-the-box default behavior.
- Q: Does `@JsonIgnore` prevent a field from being deserialized (read from incoming JSON) as well as serialized (written to outgoing JSON)? → A: By default, yes, it affects both directions — Jackson also offers `@JsonIgnoreProperties` / separate read-only (`@JsonProperty(access = READ_ONLY)`) and write-only annotations for cases needing asymmetric behavior (e.g., accepting a password on input but never returning it on output).

Coding Exercise

L1: Serialize and deserialize a custom record using Jackson's `ObjectMapper`, including a `LocalDate` field. **L2:** Add `@JsonIgnore` to a sensitive field and verify it never appears in the serialized output. **L3:** Test deserializing JSON with an extra, unrecognized field, both with and without `FAIL_ON_UNKNOWN_PROPERTIES` disabled. **L4 (Interview):** Explain why `@JsonIgnore` is a security-relevant annotation, not just a convenience. **L5 (Senior):** Review a DTO class about to be exposed

via a public API — identify any fields that should be `@JsonIgnore` d and justify each. **L6 (Mastery):** Explain, from memory, the difference between JSON serialization (Jackson) and `java.io.Serializable` , and when each is actually used.

CHAPTER 7 — Authentication Concepts

Telugu Explanation

Authentication అంటే "మీరు ఎవరో నిరూపించడం" — identity verify చేయడం. Common mechanisms: **Basic Auth** (username/password ప్రతి request తో, base64-encoded, weak without HTTPS), **Session-based** (Ch.3, login తర్వాత session cookie), **Token-based / JWT** (JSON Web Token — stateless, self-contained, cryptographically signed), **OAuth2** (third-party delegated authentication, Book 14 లో వివరంగా).

Professional English Explanation

Authentication means "proving who you are" — verifying identity. Common mechanisms: **Basic Auth** (username/password on every request, base64-encoded — weak without HTTPS, since base64 isn't encryption), **Session-based** (Ch.3, a session cookie after login), **Token-based / JWT** (JSON Web Token — stateless, self-contained, cryptographically signed), and **OAuth2** (third-party delegated authentication, covered fully in Book 14).

Java Code — Understanding a JWT's Structure

```
import java.util.Base64;

public class AuthenticationConceptsDemo {
    public static void main(String[] args) {
        // A JWT has 3 base64url-encoded parts, separated by dots: HEADER.PAYLOAD.SIGNATURE
        String sampleJwt = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9."
// header
        +
"eyJzdWIiOiJyYXZpQGV4YWlwbGUuY29tIiwicm9sZSI6IkpFETU0IiwiaXhwIjoxNzY1Mjg5NjAwfQ." //
payload
        + "SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c";
// signature

        String[] parts = sampleJwt.split("\\.");
        String header = new String(Base64.getUrlDecoder().decode(parts[0]));
        String payload = new String(Base64.getUrlDecoder().decode(parts[1]));

        System.out.println("Header (algorithm/type): " + header);
        System.out.println("Payload (claims): " + payload);
        System.out.println("Signature (base64, NOT decodable - cryptographic proof of
integrity): " + parts[2]);

        System.out.println("""

                KEY INSIGHT: The payload is only BASE64-ENCODED, not ENCRYPTED - it is fully
                readable by anyone who intercepts the token (try decoding it yourself above).
                NEVER put secrets (passwords, credit card numbers) in a JWT payload!

                The SIGNATURE is what makes a JWT trustworthy: it's a cryptographic hash of the
                header+payload, computed using a secret key ONLY the server knows. If ANY part
of
                the header or payload is tampered with, the signature verification fails, and
the
                server REJECTS the token - this is what makes it 'tamper-evident,' not
'secret.'

                """);

        System.out.println("Basic Auth header example (base64, NOT secure without HTTPS:");
        String basicAuthValue =
Base64.getEncoder().encodeToString("ravi:mypassword".getBytes());
        System.out.println("Authorization: Basic " + basicAuthValue);
        System.out.println("Decoded back (proving base64 != encryption): "
                + new String(Base64.getDecoder().decode(basicAuthValue)));
    }
}
```

Output

```
Header (algorithm/type): {"alg":"HS256","typ":"JWT"}
Payload (claims): {"sub":"ravi@example.com","role":"ADMIN","exp":1735689600}
Signature (base64, NOT decodable - cryptographic proof of integrity):
SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

KEY INSIGHT: The payload is only BASE64-ENCODED, not ENCRYPTED - it is fully readable by anyone who intercepts the token (try decoding it yourself above). NEVER put secrets (passwords, credit card numbers) in a JWT payload!

The SIGNATURE is what makes a JWT trustworthy: it's a cryptographic hash of the header+payload, computed using a secret key ONLY the server knows. If ANY part of the header or payload is tampered with, the signature verification fails, and the server REJECTS the token - this is what makes it 'tamper-evident,' not 'secret.'

Basic Auth header example (base64, NOT secure without HTTPS):
Authorization: Basic cmF2aTpteXBhc3N3b3Jk
Decoded back (proving base64 != encryption): ravi:mypassword

Internal Working

- **Password storage** (a foundational authentication concern, detailed fully in Book 14) never stores the actual password — it stores a **salted hash** (via bcrypt/Argon2/PBKDF2), verified by hashing the login attempt's password and comparing hashes; this means even if the database is breached, actual passwords aren't directly exposed (though weak/common passwords remain crackable via the hash).
- A **JWT's stateless nature** is its key architectural advantage over session-based auth (Ch.3): since the token itself carries all the claims (`sub` , `role` , `exp`) and is cryptographically verifiable, any server instance can authenticate a request without needing to look anything up in shared session storage — directly solving Ch.3's multi-server session-sharing problem, which is exactly why JWTs are the dominant choice in modern microservices architectures (Book 16).
- The `exp` (expiration) claim is critical — an unexpiring token, if ever leaked/stolen, would grant indefinite access; short-lived access tokens (often paired with a longer-lived, more carefully-guarded "refresh token" to obtain new access tokens) is the standard production pattern, fully detailed in Book 14.

Real-World Example

Telugu: Modern REST APIs `Authorization: Bearer <JWT>` header వాడతారు — login endpoint credentials verify చేసి JWT issue చేస్తుంది; తర్వాతి ప్రతి request ఆ JWT ని పంపుతుంది, server signature verify చేసి (database lookup అవసరం లేకుండా) request ని authenticate చేస్తుంది. English: Modern REST APIs use the `Authorization: Bearer <JWT>` header pattern — a login endpoint verifies credentials once and issues a JWT; every subsequent request sends that JWT, and the server verifies its signature (no database lookup needed) to authenticate the request — exactly the stateless pattern this chapter builds toward, and the foundation for Book 14's full Spring Security JWT implementation.

Interview Answer

"Authentication verifies identity. Basic Auth sends base64-encoded (not encrypted) credentials on every request — insecure without HTTPS. Session-based auth (Ch.3) uses a server-side session looked up via a cookie. JWTs are stateless, self-contained tokens with a cryptographic signature — the payload is readable by anyone (base64, not encryption) but tamper-evident, since any modification invalidates the signature. JWTs' statelessness is why they dominate modern microservices — any server can verify a token without shared session storage."

Cross Questions

- Q: Is a JWT's payload encrypted? → A: No — it's base64-encoded, which is trivially reversible by anyone; the signature provides tamper-evidence, not confidentiality, which is why secrets should never be placed in a JWT payload.
- Q: Why are JWTs particularly well-suited to microservices architectures? → A: Their statelessness means any service instance can independently verify a token's signature without a shared session store — directly avoiding Ch.3's cross-server session-sharing complexity.
- Q: Why is Basic Auth considered weak without HTTPS? → A: Base64 encoding provides no confidentiality — credentials are trivially recoverable by anyone intercepting the traffic; HTTPS's encryption is what actually protects the credentials in transit, not the base64 encoding itself.

Tricky Questions

- Q: If an attacker intercepts a valid JWT (but doesn't know the server's signing secret), can they modify its `role` claim from `USER` to `ADMIN` and have it accepted? → A: No — modifying the payload changes the header+payload bytes the signature was computed over, so the signature verification would fail on the server side, and the tampered token would be rejected; this is precisely the tamper-evidence property the signature provides.
- Q: Does a longer JWT expiration time make an application more or less secure, all else equal? → A: Less secure, generally — a longer-lived token grants a larger window of exposure if it's ever stolen; the standard mitigation is short-lived access tokens plus a separate, more carefully-protected refresh-token flow (Book 14), balancing security against the inconvenience of frequent re-authentication.

Coding Exercise

L1: Decode a sample JWT's header and payload manually (as shown), and explain why this doesn't compromise security given the signature. **L2:** Encode and decode a Basic Auth header, demonstrating that base64 provides no real confidentiality. **L3:** Research (via documentation) how bcrypt/Argon2 password hashing works at a conceptual level, and summarize why salting matters. **L4 (Interview):** Explain why a JWT is described as "tamper-evident" rather than "secret" or "encrypted." **L5 (Senior):** Design an authentication strategy (session-based vs JWT-based) for a new multi-instance, load-balanced microservice, justifying your choice. **L6 (Mastery):** Explain, from memory, why JWTs solve the cross-server session-sharing problem that Ch.3 identified for session-based auth.

CHAPTER 8 — Authorization Concepts

Telugu Explanation

Authorization అంటే "మీరు ఏమి చేయగలరో నిర్ణయించడం" — authentication తర్వాత వచ్చే step (Ch.1's 401 vs 403 distinction ఇక్కడ apply అవుతుంది). Common models: **RBAC (Role-Based Access Control)** — users కి roles (ADMIN , USER) assign చేయడం, roles కి permissions attach చేయడం; **ABAC (Attribute-Based Access Control)** — finer-grained, multiple attributes (department, resource owner, time of day) మీద ఆధారపడి decisions.

Professional English Explanation

Authorization means "deciding what you're allowed to do" — the step after authentication (this is exactly Ch.1's 401 vs 403 distinction in practice). Common models: **RBAC (Role-Based Access Control)** — users are assigned roles (ADMIN , USER), and permissions attach to roles; **ABAC (Attribute-Based Access Control)** — finer-grained decisions based on multiple attributes (department, resource ownership, time of day).

Java Code — A Simplified RBAC Model

```
import java.util.*;

public class AuthorizationConceptsDemo {

    enum Permission { READ_ORDER, CREATE_ORDER, DELETE_ORDER, MANAGE_USERS }

    enum Role {
        VIEWER(Set.of(Permission.READ_ORDER)),
        CUSTOMER(Set.of(Permission.READ_ORDER, Permission.CREATE_ORDER)),
        ADMIN(Set.of(Permission.READ_ORDER, Permission.CREATE_ORDER, Permission.DELETE_ORDER,
Permission.MANAGE_USERS));

        final Set<Permission> permissions;
        Role(Set<Permission> permissions) { this.permissions = permissions; }
    }

    record User(String username, Set<Role> roles) {
        boolean hasPermission(Permission permission) {
            return roles.stream().anyMatch(role -> role.permissions.contains(permission)); //
Book 07, Ch.7
        }
    }

    static void authorize(User user, Permission requiredPermission, Runnable action) {
        if (!user.hasPermission(requiredPermission)) {
            throw new SecurityException(user.username() + " lacks permission: " +
requiredPermission); // maps to 403 (Ch.1)
        }
        action.run();
    }

    public static void main(String[] args) {
        User viewer = new User("guest_viewer", Set.of(Role.VIEWER));
        User admin = new User("system_admin", Set.of(Role.ADMIN));

        authorize(viewer, Permission.READ_ORDER, () -> System.out.println("Viewer reading order
- OK"));

        try {
            authorize(viewer, Permission.DELETE_ORDER, () -> System.out.println("Viewer
deleting order"));
        } catch (SecurityException e) {
            System.out.println("Blocked (would map to HTTP 403): " + e.getMessage());
        }

        authorize(admin, Permission.DELETE_ORDER, () -> System.out.println("Admin deleting
order - OK"));

        // ABAC-flavored example: resource-ownership check, beyond simple role
        record Order(String id, String ownerUsername) {}
        Order someOrder = new Order("ORD-1", "ravi");
        User ravi = new User("ravi", Set.of(Role.CUSTOMER));
        User anitha = new User("anitha", Set.of(Role.CUSTOMER));

        System.out.println("ravi can view own order: " + canViewOwnOrder(ravi, someOrder));
        System.out.println("anitha can view ravi's order: " + canViewOwnOrder(anitha,
someOrder));
    }
}
```

```

    record Order(String id, String ownerUsername) {}
    static boolean canViewOwnOrder(User user, Order order) {
        // ABAC: decision depends on an ATTRIBUTE (resource ownership), not just the user's
        ROLE
        return user.hasPermission(Permission.READ_ORDER) &&
            order.ownerUsername().equals(user.username());
    }
}

```

Output

```

Viewer reading order - OK
Blocked (would map to HTTP 403): guest_viewer lacks permission: DELETE_ORDER
Admin deleting order - OK
ravi can view own order: true
anitha can view ravi's order: false

```

Internal Working

- RBAC's core value is **indirection**: permissions attach to **roles**, not directly to individual users — granting/revoking a user's access becomes a matter of assigning/removing a role, rather than editing a long list of individual permissions per user; this is a genuine, practical administrative simplification at scale (thousands of users, dozens of permissions).
- The `canViewOwnOrder` example demonstrates why **pure RBAC is sometimes insufficient**: "can view orders" (a role-level permission) doesn't distinguish "can view *any* order" from "can view *only your own* orders" — that distinction requires checking an **attribute** of the specific resource (its owner) against an attribute of the requester, which is exactly ABAC's added expressiveness over plain RBAC.
- This chapter's `SecurityException` conceptually maps to an HTTP `403 Forbidden` response (Ch.1) — the user is authenticated (Ch.7 already succeeded, we know who they are) but the authorization check fails; this two-step "authenticate, then authorize" flow is exactly what Spring Security's filter chain (Ch.4, Book 14) implements in a real application.

Real-World Example

Telugu: E-commerce లో, customer తన own orders మాత్రమే చూడగలగాలి, ఇతరుల orders కాదు — plain RBAC (role="CUSTOMER") సరిపోదు, resource ownership check (ABAC-style) కూడా అవసరం. Admin dashboards, multi-tenant SaaS applications లో ఇది extremely common requirement. English: In e-commerce, a customer must only see their own orders, not other customers' — plain RBAC (role="CUSTOMER") alone is insufficient; a resource-ownership check (ABAC-style) is also required. This exact "role permits the action type, but ownership/attribute check restricts it to specific instances" pattern is an extremely common real requirement in admin dashboards and multi-tenant SaaS applications.

Interview Answer

"Authorization decides what an authenticated user can do — the step after authentication, corresponding to the 401-vs-403 distinction from Ch.1. RBAC assigns permissions to roles, then

roles to users, simplifying administration at scale. ABAC goes further, making decisions based on resource attributes (like ownership) in addition to role — necessary whenever 'can perform this action type' isn't the same as 'can perform it on this specific resource,' like a customer only being able to view their own orders."

Cross Questions

- Q: What administrative benefit does RBAC provide over assigning permissions directly to individual users? → A: Granting/revoking access becomes a matter of assigning/removing a role rather than editing per-user permission lists — a significant simplification at scale.
- Q: Why is plain RBAC sometimes insufficient for real requirements? → A: A role-level permission like "can read orders" doesn't distinguish between reading any order versus only your own — that requires checking a resource-specific attribute (ownership), which is what ABAC adds.
- Q: How does the 401-vs-403 distinction from Ch.1 map onto authentication vs authorization? → A: 401 means authentication failed/is missing (we don't know who you are); 403 means authentication succeeded but authorization failed (we know who you are, but you're not allowed to do this).

Tricky Questions

- Q: Can a user legitimately have multiple roles simultaneously, and if so, how should permission checks combine them? → A: Yes, commonly — the standard approach (as shown in the demo's `hasPermission`) is a union: the user has a given permission if *any* of their assigned roles grants it, not requiring *all* roles to agree.
- Q: Is checking authorization purely at the API/controller layer always sufficient? → A: Not necessarily — for a genuinely secure system, authorization checks are often also needed at the service/data layer (defense in depth), since a bug or a new internal code path that bypasses the controller layer's check could otherwise expose unauthorized access; this is a real architectural consideration for security-critical systems.

Coding Exercise

L1: Extend the RBAC model with a new role and permission, and write authorization checks for it.

L2: Implement a resource-ownership (ABAC-style) check for a different domain (e.g., a document-management system). **L3:** Map each of 5 authorization failure scenarios to the correct HTTP status code (401 vs 403), justifying each. **L4 (Interview):** Explain the difference between RBAC and ABAC

with a concrete example where RBAC alone is insufficient. **L5 (Senior):** Design an authorization

model for a multi-tenant SaaS application where users can belong to multiple organizations with different roles per organization. **L6 (Mastery):** Explain, from memory, why authorization checks are sometimes needed at multiple layers (defense in depth), not just the API controller layer.

CHAPTER 9 — Backend Architecture Patterns

Telugu Explanation

Production backend applications సాధారణంగా **layered architecture** గా design అవుతాయి: **Controller/Presentation layer** (HTTP handling, Ch.1-2), **Service layer** (business logic), **Repository/DAO layer** (data access, Book 09), **Domain/Entity layer** (core business objects). ప్రతి layer క్రిందిదాని మీద మాత్రమే depend అవుతుంది, పైదాని మీద కాదు — ఇది Book 02's coupling/cohesion principles direct గా backend architecture కి apply చేయడమే.

Professional English Explanation

Production backend applications are typically designed as a **layered architecture**: **Controller/Presentation layer** (HTTP handling, Ch.1-2), **Service layer** (business logic), **Repository/DAO layer** (data access, Book 09), **Domain/Entity layer** (core business objects). Each layer depends only on the layer below it, never above — directly applying Book 02's coupling/cohesion principles to backend architecture.

Java Code — A Complete Layered Example

```
import java.util.*;

// DOMAIN LAYER: core business objects, no framework/HTTP/DB dependency
record Order(String id, String customerId, double total, String status) {}

class InsufficientStockException extends RuntimeException {           // Book 04, Ch.6
    InsufficientStockException(String message) { super(message); }
}

// REPOSITORY LAYER: data access abstraction (Book 09/06's Repository pattern), no business
// logic
interface OrderRepository {
    void save(Order order);
    Optional<Order> findById(String id);
    List<Order> findByCustomerId(String customerId);
}

class InMemoryOrderRepository implements OrderRepository {          // stand-in for a
    // real JDBC/JPA impl (Book 09/13)
    private final Map<String, Order> store = new HashMap<>();
    @Override public void save(Order order) { store.put(order.id(), order); }
    @Override public Optional<Order> findById(String id) { return
Optional.ofNullable(store.get(id)); }
    @Override public List<Order> findByCustomerId(String customerId) {
        return store.values().stream().filter(o -> o.customerId().equals(customerId)).toList();
    }
}

// SERVICE LAYER: business logic, orchestrates repositories, has NO knowledge of HTTP at all
class OrderService {
    private final OrderRepository orderRepository;
    private final Map<String, Integer> stockLevels;                 // stand-in for an
    // InventoryService

    OrderService(OrderRepository orderRepository, Map<String, Integer> stockLevels) {
        this.orderRepository = orderRepository;
        this.stockLevels = stockLevels;
    }

    Order placeOrder(String customerId, String productId, double total) {
        int stock = stockLevels.getOrDefault(productId, 0);
        if (stock <= 0) throw new InsufficientStockException("No stock for product: " +
productId); // business rule
        stockLevels.put(productId, stock - 1);

        Order order = new Order(UUID.randomUUID().toString(), customerId, total, "PLACED");
        orderRepository.save(order);
        return order;
    }

    List<Order> getOrdersForCustomer(String customerId) { return
orderRepository.findByCustomerId(customerId); }
}

// CONTROLLER LAYER: HTTP-specific, translates HTTP <=> service calls, knows NOTHING about
// business rules itself
class OrderController {
    private final OrderService orderService;
```

```

    OrderController(OrderService orderService) { this.orderService = orderService; }

    String handlePlaceOrderRequest(String customerId, String productId, double total) {
// simulates an @PostMapping
        try {
            Order order = orderService.placeOrder(customerId, productId, total);
            return "201 Created: " + order;
// maps to Ch.1's 201
        } catch (InsufficientStockException e) {
            return "409 Conflict: " + e.getMessage();
// maps to Ch.1's 409
        }
    }
}

public class BackendArchitectureDemo {
    public static void main(String[] args) {
        OrderRepository repository = new InMemoryOrderRepository();
        Map<String, Integer> stock = new HashMap<>(Map.of("P1", 1, "P2", 0));
        OrderService service = new OrderService(repository, stock);
        OrderController controller = new OrderController(service);

        System.out.println(controller.handlePlaceOrderRequest("C1", "P1", 500.0)); //
succeeds
        System.out.println(controller.handlePlaceOrderRequest("C1", "P2", 300.0)); //
fails - no stock
        System.out.println("Orders for C1: " + service.getOrdersForCustomer("C1").size());
    }
}

```

Output

```

201 Created: Order[id=..., customerId=C1, total=500.0, status=PLACED]
409 Conflict: No stock for product: P2
Orders for C1: 1

```

Internal Working & Design Rationale

- **Strict layering direction** (Controller → Service → Repository, never the reverse, and never skipping a layer) is what keeps the system testable and maintainable — `OrderService` can be fully unit-tested (Book 15) with a fake/mock `OrderRepository`, entirely independent of any HTTP server or real database; `OrderController` can be tested independent of real business logic by mocking `OrderService`.
- The **Service layer knows nothing about HTTP** — it throws a domain-specific exception (`InsufficientStockException`, Book 04, Ch.6) rather than, say, directly constructing an HTTP response; the **Controller layer** is exactly where that translation from "business outcome" to "HTTP status code + response body" happens (Ch.1's status codes) — this separation is precisely what makes the same service logic reusable behind a REST API, a message-queue consumer (Book 17), or a CLI tool without duplicating business rules in each.
- This layered structure, once you introduce Spring (Book 11-13), becomes `@RestController` → `@Service` → `@Repository` — the exact same architectural shape, now with dependency injection (Book 11) wiring the layers together instead of manual constructor calls in `main()`.

Real-World Example

Telugu: ఈ chapter యొక్క `OrderController / OrderService / OrderRepository` structure ఖచ్చితంగా Book 11-13 లో మీరు నేర్చుకోబోయే Spring `@RestController / @Service / @Repository` architecture — ఇక్కడ manual గా wire చేసిన dependencies, Spring లో `@Autowired` / constructor injection ద్వారా automatic గా జరుగుతాయి. English: This chapter's `OrderController / OrderService / OrderRepository` structure is exactly the Spring `@RestController / @Service / @Repository` architecture you'll build in Books 11-13 — the manually-wired dependencies here become automatic via `@Autowired` / constructor injection in Spring, but the architectural shape and layering discipline are identical.

Interview Answer

"Backend applications are typically layered: Controller (HTTP handling), Service (business logic), Repository (data access), Domain (core objects) — each layer depending only downward, never upward or sideways. The service layer stays HTTP-agnostic, throwing domain exceptions that the controller layer translates into appropriate status codes. This separation makes each layer independently testable and lets the same business logic be reused across different entry points (REST API, message consumer, CLI) — it's exactly the shape Spring's `@RestController / @Service / @Repository` formalizes with dependency injection."

Cross Questions

- Q: Why should the Service layer never directly construct an HTTP response or know about status codes? → A: That knowledge belongs to the Controller layer, which is the only layer that should know about HTTP at all — keeping the Service layer HTTP-agnostic lets the same business logic be reused behind non-HTTP entry points (message queues, CLI tools, batch jobs) without duplication.
- Q: How does this layered structure make unit testing easier? → A: Each layer's dependencies can be mocked/faked (e.g., testing `OrderService` with an in-memory or mock `OrderRepository`), isolating what's actually being tested from the layers above and below it (Book 15).
- Q: What Spring annotations correspond to each layer? → A: `@RestController` (Controller), `@Service` (Service), `@Repository` (Repository) — covered fully in Books 11-13, formalizing exactly this chapter's manual structure with dependency injection.

Tricky Questions

- Q: Is it ever acceptable for a Controller to call a Repository directly, skipping the Service layer? → A: Generally considered an anti-pattern — even for a trivial "just fetch and return" operation, skipping the Service layer erodes the architectural discipline over time as the codebase grows, making it harder to consistently enforce business rules, transactions (Book 09, Ch.5), and cross-cutting concerns later.
- Q: If two different Controllers (a REST API controller and, say, a scheduled batch job) both need the same business operation, where should that logic live? → A: In the Service layer, called by both — this is exactly the reusability benefit of keeping business logic out of the Controller layer,

avoiding duplicating (and potentially inconsistently maintaining) the same business rule in two places.

Coding Exercise

L1: Build a layered Controller/Service/Repository structure for a different domain (e.g., a library book-loan system). **L2:** Add a domain-specific exception in the Service layer, and translate it to the correct HTTP status code in the Controller layer. **L3:** Write a simple unit test (conceptually, using a hand-written fake) for the Service layer, independent of any Controller or real Repository. **L4 (Interview):** Explain why the Service layer should remain HTTP-agnostic. **L5 (Senior):** Review a codebase where business logic is duplicated across a REST controller and a batch job — propose a refactoring plan extracting shared logic into the Service layer. **L6 (Mastery):** Explain, from memory, how this chapter's manual layered architecture maps onto Spring's `@RestController` / `@Service` / `@Repository` annotations (previewing Books 11-13).

CHAPTER 10 — Mini Project: Servlet-Based Task Manager

Goal

Combine every concept from this book into one small, realistic servlet-based backend — no Spring yet (that's Books 11-12), pure servlets and manual wiring, to fully internalize what Spring will later automate.

Requirements

1. **Layered architecture** (Ch.9): `TaskController` (servlet) → `TaskService` (business logic) → `TaskRepository` (in-memory, or JDBC-backed via Book 09).
2. **RESTful endpoints** (Ch.5) via a single servlet mapped to `/api/tasks/*`, dispatching by HTTP method and path: `GET /api/tasks`, `GET /api/tasks/{id}`, `POST /api/tasks`, `PATCH /api/tasks/{id}`, `DELETE /api/tasks/{id}`.
3. **JSON** (Ch.6): request bodies parsed via Jackson, responses serialized via Jackson, with a `TaskDto` record separate from the internal `Task` domain object (never exposing internal-only fields).
4. **Correct status codes** (Ch.1): 200/201/204/400/404/409 used appropriately for each scenario.
5. **A logging filter** (Ch.4) recording method, path, status, and duration for every request.
6. **Session-based "current user" tracking** (Ch.3): a `/api/login` endpoint setting a session attribute; task endpoints require a valid session.
7. **Simple authentication + authorization** (Ch.7-8): reject requests without a valid session (401), and reject non-owners from modifying another user's tasks (403) using an RBAC-lite ownership check.
8. Handle at least 2 domain-specific exceptions (Book 04, Ch.6), translated to correct status codes at the controller boundary.

Concepts Reinforced

Every chapter in this book — HTTP semantics, servlet lifecycle, sessions, filters, REST design, JSON, authentication, authorization, and layered architecture — applied together in one small but structurally realistic backend application.

Stretch Goal

Swap the in-memory `TaskRepository` for a real JDBC-backed implementation (Book 09), using a connection pool (Book 09, Ch.7) and proper transaction handling (Book 09, Ch.5) for any multi-step operation.

FINAL REVISION NOTES

- **HTTP**: stateless; methods have idempotency semantics (GET/PUT/DELETE idempotent, POST typically not); status codes grouped by first digit, used correctly for tooling/client compatibility.
 - **Servlets**: one shared instance across all requests — instance fields are shared mutable state, avoid them; `init()` / `destroy()` run once, `service()` / `doGet()` / `doPost()` run per request.
 - **Sessions/cookies**: layer state on top of stateless HTTP via a session-ID cookie; `HttpOnly` / `Secure` flags are real security controls; multi-server deployments need sticky sessions or a shared store.
 - **Filters/Listeners**: filters form a chain (pass via `chain.doFilter()` or short-circuit), ideal for cross-cutting concerns; listeners react to lifecycle events. Spring Security's filter chain is built on exactly this.
 - **REST**: architectural style, not a protocol; statelessness enables horizontal scalability; resource-based URLs + correct HTTP methods let generic tooling reason correctly about requests.
 - **JSON/Jackson**: reflection-based mapping; `@JsonIgnore` is security-relevant; distinct from `java.io.Serializable` (JVM-internal binary, not JSON).
 - **Authentication**: proving identity; JWTs are stateless and tamper-evident (signed) but NOT encrypted (base64 payload is readable) — never put secrets in a JWT payload.
 - **Authorization**: deciding permitted actions (401 vs 403); RBAC (role→permissions) simplifies administration; ABAC adds resource-attribute checks (e.g., ownership) RBAC alone can't express.
 - **Layered architecture**: Controller (HTTP-aware) → Service (HTTP-agnostic business logic) → Repository (data access) → Domain; strict downward dependency direction enables testability and reuse across entry points — directly maps onto Spring's `@RestController` / `@Service` / `@Repository`.
-



CHEAT SHEET

HTTP: stateless | GET/PUT/DELETE idempotent, POST typically not | 2xx/3xx/4xx/5xx status groups
401=not authenticated | 403=authenticated but not allowed | 404=not found | 409=conflict
Servlet lifecycle: init()(once) -> service()/doGet()/doPost()(per request, SHARED instance) -> destroy()(once)
-> instance fields = shared mutable state risk, avoid them
Session: server-side state keyed by a cookie (JSESSIONID) | HttpOnly=no JS access | Secure=HTTPS only
multi-server needs sticky sessions or shared store (Redis)
Filter chain: chain.doFilter()=pass onward | no call=short-circuit | code AFTER doFilter() runs on the way back out
REST: architectural style | stateless (scalability) | resource URLs (nouns) | correct HTTP methods (uniform interface)
JSON (Jackson): reflection-based | @JsonProperty(rename) @JsonIgnore(exclude, SECURITY) @JsonFormat(dates)
!= java.io.Serializable (JVM binary, not JSON)
JWT: header.payload.signature | payload = base64 (READABLE, not encrypted) | signature = tamper-evident, not secret
NEVER put secrets in payload | short expiry + refresh token = standard pattern
RBAC: permissions on ROLES, roles on users (admin simplification) | ABAC: + resource ATTRIBUTE checks (e.g. ownership)
Layered architecture: Controller(HTTP) -> Service(business logic, HTTP-agnostic) -> Repository(data) -> Domain
strict downward dependency = testability + reuse across entry points = maps to @RestController/@Service/@Repository



INTERVIEW QUESTION BANK — Advanced Java / Backend Fundamentals

Beginner

1. What is the difference between GET and POST?
2. What is a servlet, and what are its lifecycle methods?
3. What is a cookie, and what is it used for?
4. What is JSON, and why is it used in REST APIs?
5. What is the difference between authentication and authorization?

Intermediate 6. Explain idempotency and why it matters for HTTP methods. 7. Why is a shared servlet instance's mutable state a concurrency risk? 8. What is the purpose of a Filter, and how does the filter chain work? 9. Explain what a JWT is and why its payload isn't considered secret. 10. Explain RBAC and give an example of a requirement it can't express alone.

Advanced 11. Explain REST's statelessness principle and its connection to horizontal scalability. 12. Why is `@JsonIgnore` a security-relevant annotation, not just a convenience? 13. Explain the difference between session-based and JWT-based authentication, including the multi-server trade-off. 14. Explain why the Service layer in a layered architecture should remain HTTP-agnostic. 15. Explain the difference between 401 and 403, with a concrete scenario for each.

Senior/Architect 16. Design a layered architecture (Controller/Service/Repository) for a new feature, explaining each layer's responsibility and dependency direction. 17. Design an authentication and authorization strategy for a new multi-instance microservice, justifying JWT vs session-based auth. 18. Review an API using verb-based URLs and misused HTTP methods — propose a RESTful redesign. 19. Explain how Spring's DispatcherServlet, filter chain, and Security filter chain (previewed here, detailed in Book 11/14) build on this book's servlet/filter concepts. 20. Design a resource-ownership-aware authorization model (ABAC-style) for a multi-tenant application.

(Full short/professional/deep-senior answer scaffolding for every question across the series lives in Book 23 — Java Interview Master Book.)



CROSS-QUESTION ENGINE — Sample Chains

Q: What is REST? → Q: What does statelessness mean in this context? → Q: Why does it enable horizontal scalability? → Q: What's wrong with `POST /deleteOrder?id=42`? → Q: Is REST a strict protocol or an architectural style?

Q: What is a JWT? → Q: Is it encrypted? → Q: What does the signature actually protect against? → Q: Can an attacker modify the payload and have it accepted? → Q: Why is it well-suited to microservices specifically?

Q: What is the difference between authentication and authorization? → Q: Which HTTP status code corresponds to each failing? → Q: What is RBAC? → Q: When is RBAC alone insufficient? → Q: What does ABAC add?



CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining every HTTP/servlet mechanic out loud in Telugu. **L3 — Advanced:** Build a servlet handling all 5 HTTP methods for one resource with correct status codes. **L4 — Interview:** Answer all 20 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 10 mini project fully, including the JDBC-backed stretch goal. **L6 — Mastery:** Teach Chapters 5 (REST), 7 (authentication), and 9 (layered architecture) out loud, from memory, using fresh examples.



ONE-DAY REVISION PLAN (≈5 hours)

Time	Focus
0:00–0:30	Ch.1: HTTP fundamentals — memorize the status code table
0:30–1:00	Ch.2: Servlet lifecycle — the shared-instance concurrency risk
1:00–1:30	Ch.3: Sessions/cookies
1:30–1:45	Break
1:45–2:15	Ch.4: Filters/listeners — the chain mechanism
2:15–2:45	Ch.5: REST principles
2:45–3:15	Ch.6: JSON/Jackson — the @JsonIgnore security point
3:15–3:45	Ch.7: Authentication — JWT structure, base64 vs encryption
3:45–4:15	Ch.8: Authorization — RBAC vs ABAC
4:15–4:45	Ch.9: Layered architecture — map to future Spring annotations
4:45–5:00	Interview Bank — answer all questions from memory



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1–2 (HTTP, servlets) — write and run a basic servlet if possible
2	Ch.3–4 (sessions/cookies, filters/listeners) — build a logging + auth filter
3	Ch.5–6 (REST, JSON) — redesign a non-RESTful API; serialize/deserialize with Jackson
4	Ch.7–8 (authentication, authorization) — decode a JWT; build the RBAC model
5	Ch.9 (layered architecture) — build the Controller/Service/Repository structure
6	Ch.10 + Mini Project — build the full Servlet-Based Task Manager
7	Full mock interview: all 20 bank questions + all cross-question chains, in Telugu and English, without notes



FINAL MASTERY CHECKLIST

- ☐ I can explain HTTP idempotency and correctly use status codes.
- ☐ I can explain the servlet lifecycle and the shared-instance concurrency risk.
- ☐ I can explain sessions/cookies, including HttpOnly/Secure and the multi-server sharing problem.
- ☐ I can explain the filter chain mechanism and write a working filter.
- ☐ I can explain REST's core principles and design resource-based URLs correctly.
- ☐ I can serialize/deserialize JSON with Jackson, including security-relevant annotations.
- ☐ I can explain JWT structure and why the payload isn't secret.
- ☐ I can explain RBAC vs ABAC and implement a basic authorization model.
- ☐ I can design and justify a layered backend architecture.
- ☐ I built the Servlet-Based Task Manager mini project, including the JDBC stretch goal.
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

Once every box is checked, you are ready for [11_Spring_Core.md](#) .